The collector is causally correct. The evidence that says so was never measured.

Fourth independent pass. The seal now defends code well. But the validator that certifies the data reports a number it never computes — and I proved it by getting it to bless the pre-remediation run.

FLOW_BLOCKED

One command made the smoke validator accept the run I measured *30 look-ahead violations* in, report `Future Source Violations: 0`, and stamp it with today's seal. All three round-3 criticals also remain open, unmodified.

NEW CRITICALS	R3 CRITICALS OPEN	NEW WARNINGS	TESTS PREFLIGHT RUNS	CAUSAL CODE
2	3 / 3	3	6 / 14	CORRECT

EXECUTIVE VERDICT

A widening gap between what the code does and what the framework can prove

I independently re-verified the current smoke run against the raw catalog: **2,002 rows, 2,002 exactly aligned, zero look-ahead violations.** The F3 fix is real and it holds. The collector is doing the right thing.

The problem is that *the framework cannot demonstrate this*, and reports that it has. `validate_smoke.py` guards its entire causality check behind `if "triggering_1s_ts_init" in cand_df.columns:` — and that column is never persisted. The collector passes `triggering_1s_ts_init` into `_evaluate_checkpoint()` as a parameter for a runtime `assert`, then never writes it to the record. So the branch is dead, and `future_source_violations_count: 0` plus `exact_timestamp_equality_verified: true` are **Python defaults presented in the acceptance artifact as measurements**.

That is not a theoretical concern. I pointed the validator at the June-era run containing the 30 violations I measured in round 2, and it printed `Future Source Violations: 0`, `Status: ACCEPTED`, and stamped the run with the *current* composite seal hash — code from 09:22 certifying a run from 06:46. One command, three failures: the causality check is vacuous, there is no run-to-seal binding, and the numbers in the acceptance are fabricated by default.

> **The three round-3 criticals were not addressed.** I re-ran all three verbatim; all three still land. `collect.py`'s smoke gate is byte-identical, the provenance check is still `not in (1, 2)`, and the report parser still reads two `### CRITICAL:` findings as `verdict=CLEAR, critical=0`. This round's changes went to generality instead — a `_extract_df` helper for alternative strategy attribute names, and validator version 2.

There is also a quiet structural problem underneath all of this. **Preflight runs 6 of the 14 test files that exist**, selected by a hardcoded map in `select_required_tests.py`. Every regression test written in response to rounds 1–3 — the seal tamper canaries, the round-2 invariants, the runner-collect suite, the OOS-lock suite — is outside that map and has never run under a gate. And `pytest scripts/tests/` currently *cannot collect at all*, because `test_guardrail_mutations.py` imports `scripts.check_authorized_domain`, deleted in round 3. Preflight still reports `CLEAR` with `CAUSAL_INVARIANTS` in its checklist.

BYPASS ATTEMPTS

Executed against the live repository, then reverted

NEW — THE VALIDATOR BLESSES THE PRE-REMEDIATION RUN

```
$ python scripts/validate_smoke.py --study studies/<s> \
      --run-dir runs/20260815_064656_..._day        # the run I measured 30 violations in

SMOKE VALIDATION PASSED: Gemini_clean_maturity_flip_rolling_5m_productivity
Status:                      ACCEPTED
Candidates (Target Day):     2032          <- pre-F3-fix count
Future Source Violations:    0             <- never computed; 30 measured
Exact Timestamp Equality:    True          <- never checked
Emitted Feature Count:       60 (SHA: 2a744cfa3acfa437...)
Sealed Composite Hash:       1cf24f6678df5cc8...  <- TODAY's code, on a 06:46 run
Emitted Acceptance File:     .../artifacts/smoke_acceptance.json
```

INDEPENDENT RE-MEASUREMENT OF THE SAME TWO RUNS, AGAINST THE RAW CATALOG

```
pre-fix run  (06:46, 2032 rows) : look-ahead violations = 30   (validator said 0)
current run  (14:22, 2002 rows) : look-ahead violations = 0    (genuinely correct)
```

ALL THREE ROUND-3 CRITICALS, RE-RUN VERBATIM

```
R3-1 forged smoke acceptance   -> STILL BYPASSED (reached engine build)
R3-2 hand-written status.json  -> STILL BYPASSED (seal LOCKED, provenance v1 accepted)
R3-3 prose-CRITICAL report     -> verdict=CLEAR critical=0 (STILL FAIL-OPEN)
```

THE TEST SUITE CANNOT BE COLLECTED

```
$ python -m pytest scripts/tests/ -q
ERROR collecting scripts/tests/test_guardrail_mutations.py
E   ModuleNotFoundError: No module named 'scripts.check_authorized_domain'
!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!

$ python scripts/research_preflight.py --study studies/<s>   # tests ENABLED
RESEARCH PREFLIGHT VERDICT: CLEAR (8.65s)
Checks Run: EXECUTION_MANIFEST, CAUSAL_LINT, ARTIFACT_SCHEMA,
            RESEARCH_DECISION_FIDELITY, SPEC_FIDELITY, CAUSAL_INVARIANTS
```

FINDINGS

Two criticals, three warnings, plus three carried forward

CRITICAL R4-1

The smoke validator's causality check is dead code, and its result is fabricated

`scripts/validate_smoke.py:145-159` · `strategies/flip_prediction_collector.py:400, :449-453, :622`

MECHANISM

The check is `if "triggering_1s_ts_init" in cand_df.columns:`. The collector passes `triggering_1s_ts_init=ts_avail` into `_evaluate_checkpoint()`, where it is consumed only by an `assert` at line 452 — it is never added to `cand_record`, so it cannot reach the parquet. I confirmed the column is absent from **both** the pre-fix and the current run. The guard is therefore always False, leaving `future_source_violations_count = 0` and `exact_timestamp_equality_verified = True` at their initialised values, which are then written into `smoke_acceptance.json` as findings.

FAILURE

Demonstrated above: the validator accepted a run with 30 independently measured violations and reported zero. Because `collect.py`'s smoke gate reads `deterministic_validation_verified` from this same artifact, the entire chain from "validator ran" to "full run authorized" carries a number nothing produced.

WHY MISSED

The absent-column path is silent by design — a column-presence guard is indistinguishable from a passing check in the output. And the collector's `assert triggering_1s_ts_init == T` looks like the enforcement, so the persistence gap reads as redundant rather than load-bearing.

FIX

Two lines and one guard. (1) Add `"triggering_1s_ts_init": triggering_1s_ts_init` to `cand_record` in both emission paths, and add it to the metadata column set so it does not disturb the feature contract. (2) In `validate_smoke.py`, invert the guard: `raise SmokeValidationError` when the column is *absent*, rather than skipping. A causality check that cannot run must block, not pass.

REGRESSION TEST

`test_validate_smoke_blocks_when_causality_column_absent`, and `test_validate_smoke_rejects_prefix_run` — point the validator at the 2,032-row run and assert `SmokeValidationError`. Both fail today.

CRITICAL R4-2

Nothing binds a run to the seal it is validated against

`backtests/nt_runtime/output_manager.py:64-90` · `scripts/validate_smoke.py:57-66, :80-88`

MECHANISM

`run_manifest.json` records `run_id`, `spec_sha256`, dates, instrument and telemetry — but **no composite seal hash**. The validator verifies that the seal matches the manifest *as of validation time* (a good check), then picks `sorted(runs_dir.glob("*_<study>_day"))[-1]`, or whatever `--run-dir` names, and stamps the current composite onto it. There is no field to compare, so no comparison is possible.

FAILURE

A run produced by any earlier code version can be certified against today's seal, which is precisely the staleness the smoke gate exists to prevent. Demonstrated: a 06:46 run certified with the 09:22 composite. In practice today's acceptance happens to be honest — the 14:22 run does postdate the code changes — but that is workflow discipline, not a control.

FIX

Have `OutputManager._write_initial_manifest()` record the composite from the seal it verified at launch (`collect.py` already holds `seal_data`), then have the validator require `run_manifest["composite_seal_hash"] == seal_hash` and refuse otherwise. This also closes F17 from round 1 — stale runs in `runs/` become self-identifying.

REGRESSION TEST

`test_validate_smoke_rejects_run_from_different_seal`.

WARNING R4-3

Preflight runs 6 of 14 test files, and the suite currently cannot be collected

`scripts/select_required_tests.py:23–34, :74–79` · `scripts/tests/test_guardrail_mutations.py:26`

MECHANISM

Test selection is a hardcoded path→test map plus a hardcoded default list of six. The eight files it never selects include `test_audit_seal_guard.py`, `test_round2_invariants.py`, `test_nt_runner_collect.py`, `test_spec_fidelity_and_oos_lock.py`, `test_research_preflight.py`, `test_study_factory.py`, `test_test_selection.py` and `test_guardrail_mutations.py` — that is, essentially every test written in response to this audit series. Separately, `test_guardrail_mutations.py` imports `scripts.check_authorized_domain`, which round 3 deleted, so a whole-directory pytest run aborts during collection.

FAILURE

The tamper canaries that prove the seal works have never executed under a gate. A developer running the obvious command gets `Interrupted: 1 error during collection` and no test results at all, while preflight reports `CLEAR` with `CAUSAL_INVARIANTS` listed among its checks.

WHY MISSED

Same shape as F4 in round 1 — a check whose *selection* is a hardcoded list that drifted from reality. The lint was converted to manifest-driven selection; the test selector was not.

FIX

Delete `test_guardrail_mutations.py` or repoint it at a live module. Then replace the default list with a glob of `scripts/tests/test_*.py`, and have preflight fail if the collected count is lower than the number of files on disk — the same coverage-reporting discipline that made the lint fix work.

REGRESSION TEST

`test_all_test_files_are_selectable` — assert the selector's output covers `glob("scripts/tests/test_*.py")`.

WARNING R4-4

The feature-contract check was weakened from exclusion to intersection, hiding extra columns

`backtests/nt_runtime/output_manager.py:118–119`

MECHANISM	Removing the hardcoded `meta_cols` set was the right call for study-agnosticism (F14). But the replacement is `emitted_feats = [c for c in candidates_df.columns if c in set(expected_feats)]` — an intersection. Previously the list was built by *excluding* known metadata, so any unexpected column appeared in `emitted_feats` and broke the ordered-list equality. Now unexpected columns are invisible to the check.
FAILURE	The contract still guarantees the 60 expected features are present, correct and correctly ordered — that part is sound. What it no longer detects is *additional* columns: a leaked label, an outcome field, or a debug column would pass silently. Given that this project's recorded history includes labels leaking into feature surfaces, that is the wrong direction to relax.
FIX	Keep the intersection for the ordered-identity check, and add an explicit surplus check: declare `metadata_columns` in the compiled contract and raise on any column that is in neither that set nor `expected_feats`. That keeps the study-agnosticism win and restores the strictness.
REGRESSION TEST	`test_persist_rejects_unexpected_column`.

WARNING R4-5

The causality assert is a tautology, and disappears under -O

`strategies/flip_prediction_collector.py:449-453` · `:386-401`

MECHANISM

`assert triggering_1s_ts_init == T` sits inside `_evaluate_checkpoint()`, which is only reachable after `if T > ts_avail: break` and `if T < ts_avail: continue`. By the time the call is made, `T == ts_avail` holds by construction and the argument is `ts_avail`, so the assertion cannot fail regardless of correctness. It is also removed entirely when Python runs with `-O`.

FAILURE

Not a defect on its own — the enclosing control flow is correct. The problem is that it *looks* like the causality enforcement, which is part of why the missing persistence in R4-1 went unnoticed. The real check has to happen on the emitted data, not on a value the caller just derived.

FIX

Once R4-1 persists the column, this assert becomes redundant; keep it or drop it, but do not treat it as the causality guarantee. If retained, make it a raised exception rather than an `assert` so `-O` cannot strip it.

REGRESSION TEST

Covered by R4-1's tests.

CRITICAL R3-1/2/3

Carried forward unchanged from round 3

`collect.py:41-75` · `preexec_audit_seal.py:84-87` · `run_preexec_audits.py:46-70`

R3-1	Forged `smoke_acceptance.json` — unsealed, and the gate still checks only `status`, `study_name`, `sealed_composite_sha256` and `deterministic_validation_verified`. Note the honest artifact already records `validator_file_sha256`, and I confirmed it matches the real `validate_smoke.py` hash — **the binding data is present and simply not checked**. That is a two-line fix.
R3-2	Hand-written `status.json` with `audit_provenance_version: 1` still produces `seal_status: LOCKED`. The parser remains optional; the seal never re-derives.
R3-3	A report with two `### CRITICAL:` headings plus the word "CLEAR" in unrelated prose still parses as `verdict=CLEAR, critical=0`.

VERIFIED SOUND —

What is genuinely working, and should not be disturbed

`independent measurement` · `scripts/validate_smoke.py:80-139, :162-195`

CAUSAL CODE	Re-measured against the raw catalog: the current run is **2,002 / 2,002 exactly aligned, 0 violations**. F3 holds. The skip-on-gap logic and the `obs_ts = T` anchor both survived this round's edits.
THE REST OF THE VALIDATOR	Everything other than the causality branch is real and well built: seal verification, seal-versus-manifest equality, run `status == SUCCESS`, non-zero candidates, target-date filtering in Chicago time, **rejection of any DEV/prohibited year appearing in candidate timestamps**, duplicate-key detection, feature count and ordered SHA, and both-directions coverage. Each of these would have caught a real defect.
SEAL AND CLOSURE	53-file AST closure intact; `PREEXEC_AUDIT_SEAL_VALID`; lint 46/46 at 100%. Every code-tampering bypass from rounds 1–3 remains blocked.

RECOMMENDED ORDER

Five items, none larger than a few lines

1. **R4-1** — persist `triggering_1s_ts_init` and invert the guard so an unrunnable causality check blocks. This is the one finding that makes the framework's central claim unverifiable, and it is two lines plus a guard.
2. **R4-2** — record the composite in `run_manifest.json` and require the match. Closes F17 at the same time.
3. **R3-1** — verify `validator_file_sha256` against the sealed validator, and enforce the counts the acceptance already carries. The data is there; the gate just needs to read it.
4. **R3-3, then R3-2** — require a structured `audit-summary` block; require provenance version 2 and have the seal re-parse.
5. **R4-3** — delete or repoint the broken test, glob the selector, and fail preflight when collected tests are fewer than files on disk.

A pattern worth naming, because it has now recurred four times in different places. Round 1: the lint reported CLEAR having scanned one file. Round 2: the closure test asserted seven filenames. Round 3: `coverage_pct` was the literal `100.0`. Round 4: the acceptance reports `violations: 0` from an uninitialised default. **In every case a control reported a favourable result it had not computed, and in every case the reason was that "check skipped" and "check passed" render identically.** The single most valuable convention this framework could adopt is that every check emits what it examined alongside its verdict, and that a check which cannot run raises rather than returns. The lint fix already does exactly this — it is the template.

Red team round 4, 2026-08-15 · study `Gemini_clean_maturity_flip_rolling_5m_productivity` · seal composite `1cf24f6678df5cc8…` · 53 sealed files · lint 46/46. All bypasses executed against the live working tree and reverted; `PREEXEC_AUDIT_SEAL_VALID` and clean file state reconfirmed after restore. The genuine `smoke_acceptance.json` (run 20260815_142237, 2,002 candidates) was backed up before testing and restored intact.